

MODULE 06

Supervised Learning - Regression

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Supervised Learning - Regression

What is Supervised Learning?

Supervised learning trains a model on labelled data (input-output pairs) to predict outputs for new inputs.

Regression: predicts continuous values
(house price, temperature, stock price)

Classification: predicts categories
(spam/not spam, disease/healthy)

Linear Regression

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('R2:', r2_score(y_test, y_pred))
print('RMSE:', np.sqrt(mean_squared_error(y_test, y_pred)))
print('Coefficients:', model.coef_)
print('Intercept:', model.intercept_)
```

Polynomial and Ridge Regression

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.pipeline import Pipeline

# Polynomial regression
poly = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('linear', LinearRegression())
])
poly.fit(X_train, y_train)

# Ridge regression (L2 regularisation)
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)

# Lasso regression (L1 regularisation)
lasso = Lasso(alpha=0.1)
```

```
lasso.fit(X_train, y_train)
```

Regression Evaluation Metrics

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np
```

```
MSE = mean_squared_error(y_test, y_pred)
RMSE = np.sqrt(MSE)
MAE = mean_absolute_error(y_test, y_pred)
R2 = r2_score(y_test, y_pred)
```

```
print(f'MSE: {MSE:.2f}') # Mean Squared Error
print(f'RMSE: {RMSE:.2f}') # Root Mean Squared Error
print(f'MAE: {MAE:.2f}') # Mean Absolute Error
print(f'R2: {R2:.4f}') # 1=perfect, 0=baseline
```

Advanced Regression Models

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from xgboost import XGBRegressor
```

```
# Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
# Gradient Boosting
gb = GradientBoostingRegressor(n_estimators=200, learning_rate=0.05)
gb.fit(X_train, y_train)
```

```
# XGBoost
xgb = XGBRegressor(n_estimators=300, learning_rate=0.05, max_depth=6)
xgb.fit(X_train, y_train)
```